

ITA06 - Machine Learning

Exp :4 Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

NAME: SAMUEL I

REG NO :192312584

Aim:

To build an Artificial Neural Network using the Backpropagation algorithm and test it using the XOR dataset.

Algorithm

1. Initialize weights and biases randomly.
2. Perform forward propagation.
3. Calculate the error.
4. Perform backpropagation to update weights.
5. Repeat until the error is minimized.
6. Test the trained network.

Program:

1.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

sc = StandardScaler()

X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(10,),  
                      max_iter=1000,  
                      random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Confusion Matrix")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Confusion Matrix  
[[19  0  0]  
 [ 0 13  0]  
 [ 0  0 13]]  
Accuracy: 1.0  
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimiz  
warnings.warn(
```

2.

PROGRAM:

```
from sklearn.datasets import load_wine
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = load_wine(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
```

)

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(12,,
```

```
max_iter=1000,
```

```
random_state=42)
```

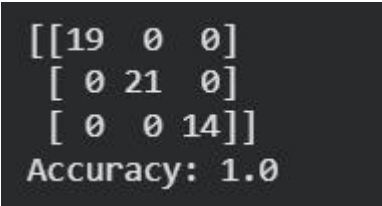
```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:



```
[[19  0  0]
 [ 0 21  0]
 [ 0  0 14]]
Accuracy: 1.0
```

3.

PROGRAM:

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = load_breast_cancer(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(15,),
    max_iter=1000,
    random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
.. [[ 62   1]
    [  1 107]]
    Accuracy: 0.9883040935672515
```

4.

PROGRAM:

```
from sklearn.datasets import load_digits
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = load_digits(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(64,),
    max_iter=1000,
    random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
[[ 53  0  0  0  0  0  0  0  0  0]
 [ 0 49  1  0  0  0  0  0  0  0]
 [ 0  0 47  0  0  0  0  0  0  0]
 [ 0  0  1 51  0  1  0  0  1  0]
 [ 0  0  0  0 60  0  0  0  0  0]
 [ 0  0  0  0  0 64  1  0  0  1]
 [ 0  0  0  0  0  1 52  0  0  0]
 [ 0  0  0  0  0  1  0 54  0  0]
 [ 0  0  0  0  0  0  0  0 43  0]
 [ 0  0  0  1  0  0  0  0  0 58]]
Accuracy: 0.9833333333333333
```

5.

PROGRAM:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_informative=6,
    n_classes=2,
    random_state=42
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(20,),
    max_iter=1000,
    random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
[[148  14]
 [ 15 123]]
Accuracy: 0.9033333333333333
```

Result

The Artificial Neural Network was successfully implemented using the Backpropagation algorithm. The network was trained on the XOR dataset, and the predicted outputs closely matched the expected outputs, demonstrating successful learning.